🚀 Demo Commands Reference - Quick Setup Guide

⚡ Pre-Demo Setup (5 minutes)

1. Start the Application

```
# Activate virtual environment
source .venv/bin/activate

# Start the server
uvicorn app:app --reload --host 0.0.0.0 --port 8000
```

2. Get Authentication Token

```
# Register a test user (if needed)
curl -X POST http://localhost:8000/auth/register \
  -H "Content-Type: application/json" \
  -d '{
    "username": "demo_user",
    "password": "demo123!",
    "email": "demo@example.com",
    "date_of_birth": "2010-01-01",
    "interests": ["machine learning", "neural networks"],
    "current_stage": "middle school",
    "study_goals": "Learn AI basics"
  }'

# Login to get JWT token
curl -X POST http://localhost:8000/auth/login \
  -H "Content-Type: application/json" \
  -d '{"username": "demo_user", "password": "demo123!"}'

# Save the JWT token from response for other commands
export JWT_TOKEN="your_jwt_token_here"
```

3. Verify System Health

```
# Check all system connections
curl -X GET http://localhost:8000/api/health \
  -H "Authorization: Bearer $JWT_TOKEN"
```

🎯 Live Demo Commands

Security Demonstration

```
# 1. Show content filtering in action
curl -X POST http://localhost:8000/api/chat \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer $JWT_TOKEN" \
  -d '{"message": "how to hack computers"}'

# 2. Show educational guardrails
curl -X POST http://localhost:8000/api/chat \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer $JWT_TOKEN" \
  -d '{"message": "tell me about cooking recipes"}'

# 3. Show inappropriate content blocking
curl -X POST http://localhost:8000/api/chat \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer $JWT_TOKEN" \
  -d '{"message": "violent content here"}'
```

RAG System Demonstration

```
# 4. Basic educational query
curl -X POST http://localhost:8000/api/chat \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer $JWT_TOKEN" \
  -d '{"message": "What is machine learning?"}'

# 5. Complex comparison query
curl -X POST http://localhost:8000/api/chat \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer $JWT_TOKEN" \
  -d '{"message": "Compare supervised and unsupervised learning"}'

# 6. Show knowledge graph integration
curl -X GET "http://localhost:8000/api/next?concept=machine%20learning" \
  -H "Authorization: Bearer $JWT_TOKEN"
```

Agentic System Demonstration

```
# 7. Get user insights from agents
curl -X GET http://localhost:8000/api/agentic/insights/YOUR_USER_ID \
  -H "Authorization: Bearer $JWT_TOKEN"

# 8. Get learning path recommendations
```

```
curl -X GET http://localhost:8000/api/agentic/path/YOUR_USER_ID \
  -H "Authorization: Bearer $JWT_TOKEN"
```

Session Continuity Demonstration

```
# 9. Check session continuity
curl -X GET http://localhost:8000/api/session-continuity \
  -H "Authorization: Bearer $JWT_TOKEN"
```

User Features Demonstration

```
# 10. Star/bookmark content
curl -X POST http://localhost:8000/api/star \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer $JWT_TOKEN" \
  -d '{"doc_id": "machine-learning__definition__c1__v1", "action":
"star"}'

# 11. Get user's starred content
curl -X GET http://localhost:8000/api/stars \
  -H "Authorization: Bearer $JWT_TOKEN"
```

Testing Demonstration

```
# 12. Run comprehensive tests
python run_comprehensive_tests.py --quick

# 13. Run specific test suites
python run_comprehensive_tests.py --suite rag_system
python run_comprehensive_tests.py --suite security
python run_comprehensive_tests.py --suite session_quiz_fixes

# 14. List all available test suites
python run_comprehensive_tests.py --list
```

Data Quality Demonstration

```
# 15. Run data quality checks
python -m pytest test_data_quality.py -v

# 16. Check Milvus collections
python -c "
from pymilvus import connections, Collection
connections.connect('default', host='YOUR_MILVUS_HOST', port='19530')
```

```
print('Rich ML Collection:', Collection('rich_ml_education').num_entities)
print('YouTube Collection:',
Collection('youtube_creator_videos').num_entities)
"
```

🎨 Frontend Demo Flow

Browser Demo Steps

1. **Open**: http://localhost:8000
2. **Register/Login**: Use demo credentials
3. **Show UI Features**:
   - Child-friendly design with teddy bear
   - Speech bubble chat interface
   - Color scheme (sky blue, sunny yellow, peachy coral)
4. **Demonstrate Session Continuity**:
   - Start conversation about "neural networks"
   - Logout and login again
   - Show session continuity modal
   - Click "Continue" and verify context preservation
5. **Show Learning Features**:
   - Ask educational questions
   - Show citations and next concepts
   - Demonstrate bookmarking functionality
   - View agentic insights in sidebar
6. **Test Protection Features**:
   - Try sending non-educational content
   - Try asking for inappropriate content
   - Show friendly error messages and educational redirects

📊 Performance Monitoring Commands

System Monitoring

```
# Check server performance
htop

# Monitor log files
tail -f app.log
```

```
# Check memory usage
free -h

# Monitor network connections
netstat -tulpn | grep :8000
```

Database Connections

```
# Test Milvus connection
python -c "from pymilvus import connections;
connections.connect('default', host='YOUR_HOST'); print('Milvus:
Connected')"

# Test Neo4j connection
python -c "from neo4j import GraphDatabase; driver =
GraphDatabase.driver('YOUR_URI', auth=('neo4j', 'password'));
print('Neo4j: Connected')"

# Test Supabase connection
python -c "from supabase import create_client; client =
create_client('YOUR_URL', 'YOUR_KEY'); print('Supabase: Connected')"
```

🔧 Troubleshooting Commands

Common Issues

```
# If Milvus connection fails
echo "Check ZILLIZ_CLOUD_URI and ZILLIZ_CLOUD_TOKEN in .env"

# If authentication fails
echo "Verify JWT_SECRET is set correctly"

# If OpenAI API fails
echo "Check OPENAI_API_KEY in .env file"

# If port 8000 is busy
lsof -ti:8000 | xargs kill -9
```

Reset Commands

```
# Clear Python cache
find . -type d -name "__pycache__" -exec rm -rf {} +

# Restart virtual environment
```

```
deactivate && source .venv/bin/activate

# Check all environment variables
env | grep -E "(OPENAI|ZILLIZ|NEO4J|SUPABASE|JWT)"
```

📝 Demo Presenter Notes

Key Points to Emphasize

- **Technical Complexity**: "This uses 8 different technologies integrated seamlessly"
- **Child Safety**: "Multiple layers of protection specifically for children"
- **AI Innovation**: "Four autonomous agents working in parallel"
- **Production Ready**: "90%+ test coverage with automated CI/CD"
- **Real-time Features**: "Live content updates every 30 minutes"

Code Files to Show

1. `app.py` - Main FastAPI application with security layers
2. `agentic_learning_system.py` - Four AI agents implementation
3. `offline_safety.py` - Child protection guardrails
4. `static/js/dashboard.js` - Session continuity frontend
5. `test_session_and_quiz_fixes.py` - Comprehensive testing
6. `.github/workflows/ci.yml` - Automated testing pipeline

Response Time Expectations

- **Health Check**: < 200ms
- **Simple Chat**: 2-3 seconds
- **Complex RAG Query**: 3-5 seconds
- **Agent Insights**: 1-2 seconds
- **Session Continuity**: < 500ms

Error Scenarios to Demo

- Quiz answer blocking (shows "A" → blocked)
- Content filtering (inappropriate content → filtered)
- Rate limiting (too many requests → throttled)
- Session recovery (logout/login → continuity offered)

This reference ensures your demo runs smoothly and showcases all the technical sophistication! 🚀⚡✨